



PROJECT REPORT

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Engineering
in
Electronics and Communication Engineering (2023-27)

Project Title:

**DESIGN, SIMULATION and FPGA IMPLEMENTATION of 8-bit SAR ADC
Controller using Verilog on PYNQ-Z2**

Submitted by:

JAYANT PUNJ – UE235057

SHUBHAM SINGH - UEM235139

Submitted to:

ABSTRACT

This project presents the design, simulation, and FPGA implementation of an 8-bit Successive Approximation Register Analog-to-Digital Converter (SAR ADC) controller. The primary objective of the project is to understand and implement the binary-search based conversion algorithm that forms the core of a SAR ADC, and to realise it both as a behavioural simulation model and as a synthesizable hardware design on the PYNQ-Z2 FPGA development board.

The behavioral simulation model was developed in Vivado using SystemVerilog. It consists of five sub-modules: a sample-and-hold circuit, a digital-to-analog converter (DAC) model, a comparator model, the SAR controller finite state machine (FSM), and a top-level integration module. A comprehensive testbench was written to verify the complete conversion process for two representative analog input values: $V_{in} = 11.2\text{ V}$ and $V_{in} = 6.5\text{ V}$, with a reference voltage of $V_{ref} = 16\text{ V}$ and 8-bit resolution (256 quantisation levels, $LSB = 0.0625\text{ V}$). Waveform simulation in Vivado confirmed bit-by-bit SAR operation, with the staircase approximation of the DAC clearly visible in the waveform. The expected digital output codes were verified: 179 ($0xB3 = 10110011$) for $V_{in} = 11.2\text{ V}$ and 104 ($0x68 = 01101000$) for $V_{in} = 6.5\text{ V}$.

Since the real data type used in SystemVerilog for floating-point values is not synthesizable on an FPGA, a separate hardware-compatible top module was developed in synthesizable Verilog for FPGA deployment. This module implements the SAR controller logic and demonstrates the conversion output using the on-board switches, push-buttons, and LEDs of the PYNQ-Z2 board. The Vivado design flow — including synthesis, implementation, and bitstream generation — was completed successfully for the device xc7z020clg400-1, confirming that the hardware design is functionally and structurally correct. A pin constraint file (XDC) was written to map the design signals to the appropriate physical pins of the PYNQ-Z2 board.

Additionally, a theoretical extension of the design to 12-bit resolution is discussed. For a 12-bit ADC with $V_{ref} = 1\text{ V}$, the LSB is $244\text{ }\mu\text{V}$, the half- LSB is $122\text{ }\mu\text{V}$, the ideal signal-to-noise ratio (SNR) is approximately 74 dB, and the effective number of bits (ENOB) for a SINAD of 68 dB is approximately 11 bits. The report covers all design parameters, mathematical calculations, module-level explanations, waveform analysis, FPGA implementation details, and a discussion of future scope including integration with a real external DAC and comparator, and extension to a 12-bit hardware ADC system.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported and guided me throughout this project. First and foremost, I am deeply grateful to my project supervisor, Department of Electronics and Communication Engineering, UIET, Panjab University, Chandigarh, for providing invaluable guidance, continuous encouragement, and insightful technical advice throughout the course of this project.

I extend my heartfelt thanks to the Head of the Department of Electronics and Communication Engineering for providing the necessary laboratory facilities and academic resources that made this work possible. I am also grateful to all the faculty members of the department for their constant support and motivation. My thanks also go to the technical staff and lab assistants for their help in facilitating access to Xilinx Vivado software, the PYNQ-Z2 FPGA development board, and associated hardware infrastructure.

I am grateful to Panjab University, Chandigarh, for providing a high-quality academic environment that fosters research and hands-on learning in advanced engineering disciplines. The university's well-resourced laboratories and access to industry-standard EDA tools such as Xilinx Vivado 2025 were instrumental in the successful completion of this project. I also acknowledge the authors of the reference materials, textbooks, research papers, and technical documentation that I consulted during this project.

SHUBHAM SINGH
UEM235139

JAYANT PUNJ
UE235057

Department of ECE, UIET
Panjab University, Chandigarh

TABLE OF CONTENTS

(Auto-generate Table of Contents in Microsoft Word using References → Table of Contents after opening this document. All chapter headings are formatted as Heading 1, 2, and 3 styles for automatic TOC generation.)

Chapter No.	Content	Page No
1	Introduction	4
2	Overview of ADC Architecture	11
3	SAR ADC Architecture	14
4	All mathematical calculations for both 8-bit and 12-bit extension, including the full bit-by-bit breakdown	18
5	All 8 modules explained (purpose, I/O, simulation vs synthesizable distinction)	23
6	Complete step-by-step waveform interpretation for both test cases	28
7	PYNQ-Z2 board, hardware demo design, and full pin constraints table	32
8	Synthesis → Implementation → Bitstream flow with WNS/TNS explanation	35
9	Results table	38
10	Applications, merits/demerits, future scope, and conclusion	41
11	Appendices	45

LIST OF FIGURES

Figure 3.1: SAR ADC Block Diagram (described in Chapter 3)

Figure 6.1: Vivado Behavioral Simulation Waveform for $V_{in} = 11.2$ V (Chapter 6)

Figure 6.2: Vivado Behavioral Simulation Waveform for $V_{in} = 6.5$ V (Chapter 6)

Figure 7.1: PYNQ-Z2 Board Overview (Chapter 7)

Figure 8.1: Vivado FPGA Design Flow (Chapter 8)

LIST OF TABLES

Table 2.1: Comparative Summary of Major ADC Architectures (Chapter 2)

Table 3.1: 4-bit SAR ADC Binary Search Example (Chapter 3)

Table 4.1: Design Parameters for 8-bit Simulation and 12-bit Extension (Chapter 4)

Table 6.1: Simulation Signals and Their Descriptions (Chapter 6)

Table 7.1: PYNQ-Z2 Pin Constraints for SAR ADC FPGA Demo (Chapter 7)

Table 9.1: Expected vs. Simulated Results for Both Test Cases (Chapter 9)

LIST OF ABBREVIATIONS AND SYMBOLS

Abbreviation / Symbol	Full Form / Meaning
ADC	Analog-to-Digital Converter
DAC	Digital-to-Analog Converter
SAR	Successive Approximation Register
FPGA	Field-Programmable Gate Array
SoC	System on Chip
HDL	Hardware Description Language
FSM	Finite State Machine
MSB	Most Significant Bit
LSB	Least Significant Bit
V _{ref}	Reference Voltage
V _{in}	Analog Input Voltage
V _{dac}	DAC Output Voltage
SNR	Signal-to-Noise Ratio
SINAD	Signal-to-Noise and Distortion Ratio
ENOB	Effective Number of Bits
DNL	Differential Non-Linearity
INL	Integral Non-Linearity
THD	Total Harmonic Distortion
SFDR	Spurious-Free Dynamic Range
LUT	Look-Up Table
FF	Flip-Flop
XDC	Xilinx Design Constraints
PL	Programmable Logic
PS	Processing System
GPIO	General Purpose Input/Output
WNS	Worst Negative Slack

Abbreviation / Symbol	Full Form / Meaning
TNS	Total Negative Slack
DRC	Design Rule Check
S&H	Sample and Hold
CLK	Clock
SV	SystemVerilog
N	Number of bits (resolution)
2^N	Number of quantisation levels
μV	Microvolts
dB	Decibels
MHz	Megahertz
ns	Nanoseconds
PYNQ	Python-based open-source hardware platform (Xilinx)

CHAPTER 1: INTRODUCTION

1.1 Background

In modern electronic systems, the interface between the physical analog world and the digital processing domain is one of the most critical engineering challenges. Physical quantities such as temperature, pressure, sound, light intensity, velocity, and biomedical signals are inherently analog in nature — they vary continuously in time and amplitude. However, digital processors, microcontrollers, FPGAs, and DSP systems operate exclusively on discrete binary data. The component that bridges this fundamental divide is the Analog-to-Digital Converter (ADC).

An ADC is a mixed-signal circuit that samples an analog signal at regular time intervals and converts each sample into a binary digital code that best represents the instantaneous amplitude of the analog signal. The accuracy of this representation depends on the resolution (number of bits), sampling rate, reference voltage, and the specific architecture of the ADC. The reverse operation — converting a digital code back to an analog signal — is performed by a Digital-to-Analog Converter (DAC).

1.2 Importance of Analog-to-Digital Conversion

The proliferation of digital systems in every area of engineering and science has made high-quality ADC design an indispensable skill for electronics engineers. ADCs are found in virtually every electronic system that interacts with the physical world:

- Embedded systems and microcontroller-based instruments use ADCs to read sensor outputs.
- Audio systems use ADCs to digitise sound waves at 44.1 kHz or higher for CD-quality audio.
- Medical devices such as ECG machines, pulse oximeters, and EEG instruments depend on ADCs to capture biological signals.
- Software-defined radio (SDR) systems use high-speed ADCs to digitise radio-frequency signals.
- Industrial measurement and control systems use ADCs to monitor physical parameters in real time.

- IoT (Internet of Things) devices use low-power ADCs to digitise data from environmental sensors before transmission.

The choice of ADC architecture significantly affects the power consumption, speed, resolution, area, and cost of the overall system. Engineers must therefore carefully select the appropriate ADC type for each application.

1.3 What is a SAR ADC?

The Successive Approximation Register ADC (SAR ADC) is one of the most widely used ADC architectures in practice. It is named after its central algorithmic component — the Successive Approximation Register — which implements a binary search algorithm to determine the digital code that best represents the analog input.

The SAR ADC operates by making a series of binary comparisons, one per bit per conversion cycle. Starting from the most significant bit (MSB) and working down to the least significant bit (LSB), the SAR controller sets each bit of the digital output register, compares the resulting DAC output to the analog input, and either retains or resets the bit based on the comparison result. This process produces a complete N-bit digital code in exactly N clock cycles (or N+1 cycles including the sampling phase).

SAR ADCs are highly valued in engineering because they offer an excellent trade-off between speed, resolution, power consumption, and architectural simplicity. They are well-suited to moderate-speed applications requiring 8 to 18 bits of resolution at sampling rates from a few kilosamples per second (ksps) up to several megasamples per second (Msps). Their low power consumption makes them ideal for battery-powered and IoT applications.

1.4 Motivation for this Project

This project was undertaken with the following primary objectives:

- To study and understand the SAR ADC conversion algorithm through a detailed behavioral simulation using SystemVerilog in Xilinx Vivado.
- To design and verify each functional block of the SAR ADC — including the sample-and-hold circuit, DAC model, comparator model, and SAR controller FSM — through modular HDL design.

- To verify the digital output codes and waveforms for representative input values through comprehensive testbench simulation.
- To adapt the design for FPGA synthesis by creating a hardware-compatible Verilog module, distinguishing between simulation-only and synthesizable constructs.
- To complete the Vivado FPGA design flow — synthesis, implementation, bitstream generation — on the PYNQ-Z2 board (device: xc7z020clg400-1).
- To analyse the limitations of the 8-bit design and discuss a theoretical extension to 12-bit resolution, including performance metrics such as SNR, ENOB, DNL, and INL.

1.5 Organisation of the Report

This report is organised as follows. Chapter 2 provides a literature review and background theory covering the major ADC architectures and a comparative analysis. Chapter 3 explains the SAR ADC architecture in detail. Chapter 4 presents the mathematical design and calculations for 8-bit and 12-bit configurations. Chapter 5 describes the Verilog/SystemVerilog design methodology and explains each module. Chapter 6 covers the behavioral simulation in Vivado with waveform interpretation. Chapter 7 explains the FPGA implementation on the PYNQ-Z2 board. Chapter 8 describes the synthesis, implementation, and bitstream generation process. Chapter 9 presents the results and discussion. Chapter 10 discusses applications, merits, demerits, and future scope. Chapter 11 concludes the report.

CHAPTER 2: LITERATURE REVIEW AND BACKGROUND THEORY

2.1 Overview of ADC Architectures

Analog-to-Digital Converters have been developed in a wide variety of architectures, each optimised for different combinations of speed, resolution, power consumption, and circuit complexity. The most common ADC types encountered in modern electronics are described below.

2.2 Flash ADC

The Flash ADC, also called the parallel ADC, is the fastest ADC architecture. It employs $2^N - 1$ comparators, where N is the resolution in bits. Each comparator compares the analog input to a unique reference voltage level, and a priority encoder converts the thermometer-coded comparator outputs into a binary code.

The Flash ADC produces a result in a single clock cycle, making it suitable for very high-speed applications such as oscilloscopes, radar, and high-frequency communications. However, the exponential growth in the number of comparators with resolution makes Flash ADCs impractical for resolutions above 8 bits. A 10-bit Flash ADC requires 1023 comparators, resulting in very high power consumption and large chip area. Offset mismatches between comparators also degrade linearity at high resolutions [1].

2.3 Counter / Single-Slope (Ramp) ADC

The counter ADC works by resetting a counter and a DAC to zero, then incrementing the counter on each clock cycle while the DAC output ramps upward. When the DAC output crosses the analog input, the counter value is latched as the digital output. This architecture is extremely simple and easy to implement but requires up to 2^N clock cycles per conversion, making it unacceptably slow for all but the most basic applications. For an 8-bit counter ADC, conversion can take up to 256 clock cycles [2].

2.4 Dual-Slope ADC

The dual-slope (integrating) ADC is a high-accuracy architecture used primarily in digital multimeters and precision instrumentation. The conversion proceeds in two phases: first, the

analog input is integrated for a fixed time period; then the integrator output is discharged at a fixed rate using a reference voltage, and the discharge time is measured.

Dual-slope ADCs are inherently immune to certain noise sources because the averaging effect of integration rejects periodic interference. They achieve high resolution (up to 20+ bits) but are very slow, with conversion times in the millisecond range. They are therefore unsuitable for fast data acquisition applications [3].

2.5 Sigma-Delta ADC

The Sigma-Delta ($\Sigma\Delta$) ADC uses oversampling and noise shaping to achieve very high resolution at low-to-moderate speeds. The converter samples the input at a rate many times higher than the Nyquist frequency (the oversampling ratio, or OSR), and a digital filter and decimator are used to extract the high-resolution digital output. Sigma-delta ADCs are widely used in audio applications (16-to-24-bit resolution) and precision measurement systems (16-to-32-bit resolution).

Their main drawbacks are high latency (due to the digital filter pipeline) and relatively low conversion speed per unit of throughput. They require sophisticated digital post-processing and are not suitable for high-frequency input signals without appropriate anti-aliasing [4].

2.6 Successive Approximation Register (SAR) ADC

The SAR ADC is the subject of this project. It implements a binary search algorithm using N clock cycles per conversion (where N is the bit resolution). It combines moderate speed, moderate-to-high resolution (8 to 18 bits), low power consumption.

SAR ADCs are found in applications ranging from microcontroller embedded ADCs (8-to-12-bit, ksps range) to precision data acquisition systems (16-to-18-bit, hundreds of ksps) and even high-speed industrial ADCs (12-to-16-bit, Msps range). The Analog Devices AD7685 and Texas Instruments ADS8681 are examples of state-of-the-art SAR ADCs with 16-bit resolution [5].

2.7 Comparative Analysis of ADC Architectures

Parameter	Flash	Counter/Ramp	Dual-Slope	Sigma-Delta	SAR
Speed	Very High	Very Low	Very Low	Low-Moderate	Moderate
Resolution (bits)	4–8	8–12	12–20	16–32	8–18
Power	Very High	Low	Low	Moderate	Low
Complexity	High	Very Low	Moderate	High	Moderate
Latency	1 cycle	2^N cycles	Milliseconds	High (pipeline)	N cycles
Area	Large	Small	Moderate	Large	Moderate
Typical Applications	Oscilloscopes, Radar	Simple meters	DMMs, Instruments	Audio, Precision	Sensors, IoT, MCUs

Table 2.1: Comparative Summary of Major ADC Architectures

2.8 Why SAR ADC for this Project?

The SAR ADC was selected for this project for the following reasons:

- Its binary-search algorithm is straightforward to implement as a finite state machine (FSM) in Verilog/SystemVerilog, making it an ideal candidate for FPGA-based digital design.
- The conversion process — testing one bit per clock cycle — is deterministic and easy to verify through waveform simulation.
- Its moderate power consumption makes it relevant to modern low-power embedded and IoT design trends.
- The modular architecture (S&H, DAC, comparator, SAR controller) lends itself naturally to hierarchical HDL design methodology.

Several published works and textbooks confirm the suitability of SAR ADC for FPGA-based controller implementation [6, 7, 8]. The digital core of a SAR ADC — the SAR controller — is entirely digital and is a natural fit for synthesis on a Zynq FPGA such as the one present on the PYNQ-Z2 board.

CHAPTER 3: SAR ADC ARCHITECTURE

3.1 Overview of SAR ADC Block Diagram

The SAR ADC consists of five principal functional blocks, as illustrated conceptually in Figure 3.1. These are:

- Sample-and-Hold (S&H) Circuit
- Digital-to-Analog Converter (DAC)
- Comparator
- Successive Approximation Register (SAR) Controller
- Control Logic (Clock, Start, Done)

The analog input signal V_{in} is first sampled and held constant by the S&H circuit. The SAR controller then performs a binary search by setting the DAC to different trial values and comparing the DAC output to the held sample. After N iterations (where N = number of bits), the digital output register contains the final converted code.

3.2 Sample-and-Hold Circuit

The sample-and-hold (S&H) circuit is the first stage of the SAR ADC. Its function is to capture the instantaneous value of the continuously varying analog input at the moment of conversion and hold it constant throughout the entire conversion period (N clock cycles). Without the S&H, the input would change during the binary search, making accurate conversion impossible.

In hardware, the S&H typically consists of a switch (MOSFET) and a hold capacitor. When the switch is closed (sample phase), the capacitor charges to the input voltage. When the switch opens (hold phase), the voltage across the capacitor remains approximately constant. In the behavioral simulation model developed in this project, the S&H is modelled as a register that captures the real-valued input when the start signal is asserted.

3.3 Digital-to-Analog Converter (DAC)

The DAC is a critical component of the SAR ADC feedback loop. During each conversion step, the SAR controller sets a trial digital code in the DAC, which converts it to an analog voltage V_{dac} . This V_{dac} is compared to the held analog input $V_{sampled}$ by the comparator.

The DAC must be at least as accurate as the desired ADC resolution to avoid degrading the linearity of the overall ADC. For an N-bit SAR ADC, the DAC must have better than 0.5 LSB settling accuracy within one clock period. Common DAC architectures used in SAR ADCs include the capacitive DAC (charge redistribution), the R-2R ladder network, and the current steering DAC. In this project's behavioral simulation, the DAC is modelled mathematically as:

$$V_{dac} = (\text{digital_out} / 2^N) \times V_{ref}$$

where digital_out is the current trial code, $N = 8$, and $V_{ref} = 16 \text{ V}$.

3.4 Comparator

The comparator is a high-gain differential amplifier used to determine whether the DAC output is greater than or less than the held analog sample. The comparator output comp_out is:

$$\text{comp_out} = 1 \text{ if } V_{in} \geq V_{dac} \text{ (current bit is retained, set to 1)}$$

$$\text{comp_out} = 0 \text{ if } V_{in} < V_{dac} \text{ (current bit is reset to 0)}$$

In a real hardware SAR ADC, the comparator is the speed-limiting element. Its decision must settle within a single clock period, and its offset voltage must be less than 0.5 LSB to avoid conversion errors. In the behavioral model, the comparator is implemented using a simple real-valued conditional comparison.

3.5 SAR Controller

The SAR controller is the heart of the SAR ADC. It is a digital finite state machine (FSM) that implements the binary search algorithm. The controller maintains the N-bit digital output register (digital_out or SAR register) and sequences the conversion process bit by bit from MSB to LSB.

The SAR controller operates as follows on each clock cycle during conversion:

- Set the current bit (starting from MSB) to 1, leaving all lower bits at their previously determined values.
- Wait one clock cycle for the DAC to settle and the comparator to make a decision.
- Read the comparator output: if $\text{comp_out} = 1$, retain the current bit (it remains 1); if $\text{comp_out} = 0$, reset the current bit to 0.
- Move to the next lower bit and repeat until all N bits have been determined.
- Assert the done signal to indicate that the conversion is complete and digital_out holds the valid result.

3.6 Binary Search Operation: 4-bit Example

To make the binary search operation concrete, consider a 4-bit SAR ADC with $V_{\text{ref}} = 16 \text{ V}$ and $V_{\text{in}} = 11.2 \text{ V}$. The $\text{LSB} = 16 / 16 = 1 \text{ V}$.

Step	Bit Tested	Trial Code (binary)	V _{dac} (V)	V _{in} vs V _{dac}	Decision	SAR Register
1	Bit 3 (MSB)	1000	8.0	$11.2 > 8.0$	Keep (1)	1000
2	Bit 2	1100	12.0	$11.2 < 12.0$	Reset (0)	1000
3	Bit 1	1010	10.0	$11.2 > 10.0$	Keep (1)	1010
4	Bit 0 (LSB)	1011	11.0	$11.2 > 11.0$	Keep (1)	1011

Table 3.1: 4-bit SAR ADC Binary Search Example ($V_{\text{in}} = 11.2 \text{ V}$, $V_{\text{ref}} = 16 \text{ V}$)

The final code 1011 (decimal 11) corresponds to a reconstructed voltage of $11 \times 1.0 \text{ V} = 11.0 \text{ V}$. The quantisation error is $11.2 - 11.0 = 0.2 \text{ V}$, which is within $0.5 \text{ LSB} = 0.5 \text{ V}$. This confirms correct ADC operation.

3.7 Generalisation to 8-bit and 12-bit

For an 8-bit SAR ADC with $V_{\text{ref}} = 16 \text{ V}$, the process described above extends to 8 steps (bits 7 through 0). The $\text{LSB} = 16 / 256 = 0.0625 \text{ V}$, and the conversion requires exactly 8 clock

cycles per conversion (plus additional cycles for sampling and output latching, depending on implementation). The complete conversion process for $V_{in} = 11.2 \text{ V}$ is detailed in Chapter 4.

For a 12-bit extension with $V_{ref} = 1 \text{ V}$, the same algorithm applies but requires 12 clock cycles. The LSB shrinks to $244 \mu\text{V}$, placing much stricter requirements on the DAC settling accuracy, comparator offset, and noise performance. All performance parameters for the 12-bit case are calculated in Chapter 4.

3.8 Clock and Timing

The clock drives the SAR controller FSM. A conversion begins when the start signal is asserted. The SAR controller operates synchronously on the rising edge of the clock. The done signal is asserted at the end of the conversion, indicating that digital out is valid. In this project's simulation, a 10 ns clock period (100 MHz) was used. In the FPGA implementation, the system clock from the PYNQ-Z2 board is used directly.

CHAPTER 4: MATHEMATICAL DESIGN AND CALCULATIONS

4.1 Basic ADC Parameters

4.1.1 Resolution

The resolution of an ADC is the number of distinct output codes it can produce. For an N-bit ADC:

$$\text{Resolution} = N \text{ bits}$$

$$\text{Number of quantisation levels} = 2^N$$

For this project: $N = 8$ bits, giving $2^8 = 256$ levels.

4.1.2 LSB Size

The Least Significant Bit (LSB) is the smallest voltage increment that the ADC can resolve. It is the voltage change corresponding to a change of 1 in the digital output code:

$$\text{LSB} = V_{\text{ref}} / 2^N$$

For the 8-bit simulation: $\text{LSB} = 16 / 256 = 0.0625 \text{ V}$

4.1.3 Quantisation Error

Quantisation error (or quantisation noise) is the inherent error introduced by representing a continuous analog signal with a finite number of discrete levels. For a uniform quantiser:

$$\text{Maximum quantisation error} = \pm 0.5 \times \text{LSB} = \pm 0.03125 \text{ V (for 8-bit, } V_{\text{ref}} = 16 \text{ V)}$$

This means that any analog input value will be represented by a digital code whose reconstructed voltage differs from the true input by at most half an LSB.

4.2 DAC Transfer Equation

The DAC converts the N-bit digital trial code D to an analog voltage V_{dac} according to:

$$V_{\text{dac}} = (D / 2^N) \times V_{\text{ref}}$$

where D is the current digital code (0 to $2^N - 1$) and V_{ref} is the reference voltage. For the 8-bit simulation with $V_{\text{ref}} = 16 \text{ V}$:

$$V_{dac} = (D / 256) \times 16 = D \times 0.0625 \text{ V}$$

4.3 Digital Output Code Equation

The digital output code D produced by the ADC for an analog input V_{in} is:

$$D = \text{floor}(V_{in} / \text{LSB}) = \text{floor}(V_{in} \times 2^N / V_{ref})$$

where $\text{floor}()$ denotes rounding down to the nearest integer (truncation to the lower code boundary). Some implementations may use rounding to the nearest integer instead.

4.4 Output Code Calculations

4.4.1 For $V_{in} = 11.2 \text{ V}$ ($V_{ref} = 16 \text{ V}$, $N = 8$)

$$\text{LSB} = 16 / 256 = 0.0625 \text{ V}$$

$$D = 11.2 / 0.0625 = 179.2$$

Since partial codes are not possible, the ADC rounds to either 179 or 180, depending on whether truncation or nearest-value rounding is used.

For $D = 179$:

- Binary: 10110011
- Hexadecimal: 0xB3
- Reconstructed voltage: $179 \times 0.0625 = 11.1875 \text{ V}$
- Quantisation error: $11.2 - 11.1875 = 0.0125 \text{ V}$ ($= 0.2 \text{ LSB}$)

For $D = 180$:

- Binary: 10110100
- Hexadecimal: 0xB4
- Reconstructed voltage: $180 \times 0.0625 = 11.25 \text{ V}$
- Quantisation error: $11.25 - 11.2 = 0.05 \text{ V}$ ($= 0.8 \text{ LSB}$)

The behavioral simulation produces $D = 179$ ($0xB3 = 10110011$) due to the $\text{floor}()$ truncation used in the comparator model.

4.4.2 For $V_{in} = 6.5 \text{ V}$ ($V_{ref} = 16 \text{ V}$, $N = 8$)

$$D = 6.5 / 0.0625 = 104.0$$

$D = 104$ exactly (no rounding needed)

- Binary: 01101000
- Hexadecimal: 0x68
- Reconstructed voltage: $104 \times 0.0625 = 6.5 \text{ V}$ (exact)
- Quantisation error: 0 V

4.5 Conversion Time and Sampling Rate

The conversion time T_{conv} is the time required to complete one N-bit conversion:

$$T_{conv} = N \times T_{clk} = N / f_{clk}$$

For $N = 8$ bits and $f_{clk} = 100 \text{ MHz}$ ($T_{clk} = 10 \text{ ns}$):

$$T_{conv} = 8 \times 10 \text{ ns} = 80 \text{ ns}$$

The maximum theoretical sampling rate (neglecting S&H acquisition time and overhead cycles) is:

$$f_{s_max} = 1 / T_{conv} = 1 / 80 \text{ ns} = 12.5 \text{ Msps}$$

In practice, additional clock cycles are needed for sampling and latching, reducing the effective throughput. A typical rule of thumb is to add 2 to 3 overhead cycles, giving $f_s \approx 9$ to 11 Msps for $f_{clk} = 100 \text{ MHz}$, $N = 8$.

4.6 Quantisation Noise and SNR

For a full-scale sinusoidal input signal, the theoretical signal-to-noise ratio (SNR) of an ideal N-bit ADC is:

$$\text{SNR}_{ideal} = 6.02 \times N + 1.76 \text{ dB}$$

$$\text{For } N = 8 \text{ bits: } \text{SNR}_{ideal} = 6.02 \times 8 + 1.76 = 49.92 \text{ dB} \approx 50 \text{ dB}$$

$$\text{For } N = 12 \text{ bits: } \text{SNR}_{ideal} = 6.02 \times 12 + 1.76 = 74 \text{ dB}$$

4.7 SINAD, ENOB, DNL, and INL

4.7.1 SINAD (*Signal-to-Noise and Distortion Ratio*)

SINAD is the ratio of the RMS signal to the RMS sum of all noise and distortion components (including harmonics). $\text{SINAD} \leq \text{SNR}_{\text{ideal}}$ in a real ADC due to non-linearity and noise. For the 12-bit design discussion, a SINAD of 68 dB is assumed.

4.7.2 ENOB (*Effective Number of Bits*)

ENOB accounts for all noise and distortion sources and gives the effective resolution of the ADC in bits:

$$\text{ENOB} = (\text{SINAD} - 1.76) / 6.02$$

For SINAD = 68 dB: $\text{ENOB} = (68 - 1.76) / 6.02 = 66.24 / 6.02 \approx 11.0$ bits

This means a 12-bit ADC with SINAD = 68 dB performs equivalently to an ideal 11-bit ADC.

4.7.3 DNL (*Differential Non-Linearity*)

DNL measures the deviation of each actual code width from the ideal 1 LSB code width:

$$\text{DNL}[k] = (\text{actual code width at code } k / \text{ideal code width}) - 1 \text{ LSB}$$

For the design target: $\text{DNL} \leq \pm 0.5 \text{ LSB}$. A DNL exceeding +1 LSB would indicate a missing code.

4.7.4 INL (*Integral Non-Linearity*)

INL measures the cumulative deviation of the ADC transfer function from the ideal straight line:

$$\text{INL}[k] = \sum \text{DNL}[i] \text{ for } i = 0 \text{ to } k$$

For the design target: $\text{INL} \leq \pm 1 \text{ LSB}$.

4.8 Comparator and DAC Specifications

4.8.1 Comparator Offset

The comparator offset voltage V_{os} introduces a systematic error in the comparison. To maintain $\leq 0.5 \text{ LSB}$ total error, the comparator offset must satisfy:

$$|V_{os}| < 0.5 \times \text{LSB}$$

For 8-bit, $V_{ref} = 16 \text{ V}$: $|V_{os}| < 0.5 \times 0.0625 \text{ V} = 31.25 \text{ mV}$

For 12-bit, $V_{ref} = 1 \text{ V}$: $|V_{os}| < 0.5 \times 244 \mu\text{V} = 122 \mu\text{V}$

4.8.2 DAC Settling Error

The DAC must settle to within 0.5 LSB of its final value within each clock period. For the 12-bit design:

$$\text{DAC settling error} < 0.5 \times \text{LSB} = 0.5 \times 244 \mu\text{V} = 122 \mu\text{V}$$

4.9 12-bit Extension Summary

Parameter	8-bit Design (Simulated)	12-bit Extension (Discussed)
Resolution (N)	8 bits	12 bits
V_{ref}	16 V	1 V
Number of levels (2^N)	256	4096
LSB size	$0.0625 \text{ V} = 62.5 \text{ mV}$	$244 \mu\text{V}$
Half LSB (0.5 LSB)	31.25 mV	$122 \mu\text{V}$
Ideal SNR	$49.92 \text{ dB} \approx 50 \text{ dB}$	74 dB
ENOB (for SINAD = 68 dB)	—	$\approx 11.0 \text{ bits}$
DNL target	$\leq \pm 0.5 \text{ LSB}$	$\leq \pm 0.5 \text{ LSB}$
INL target	$\leq \pm 1 \text{ LSB}$	$\leq \pm 1 \text{ LSB}$
Comparator offset budget	$< 31.25 \text{ mV}$	$< 122 \mu\text{V}$
DAC settling budget	$< 31.25 \text{ mV}$	$< 122 \mu\text{V}$
Conversion cycles	8	12

Table 4.1: Design Parameters for 8-bit Simulation and 12-bit Extension

CHAPTER 5: VERILOG/SYSTEMVERILOG DESIGN METHODOLOGY

5.1 Design Philosophy

The complete SAR ADC system is divided into seven design files: five files for behavioral simulation, one file for FPGA synthesis, and one XDC constraint file for pin assignments. The key distinction is that the behavioral simulation files use the SystemVerilog real data type for floating-point arithmetic, which is valid for simulation but is not synthesizable on an FPGA. The FPGA implementation file uses only synthesizable Verilog constructs (registers, wires, logic gates, and integer arithmetic).

5.2 Module 1: sample_hold.sv (Behavioral Simulation)

Purpose and Function

The sample_hold module models the sample-and-hold circuit. When the start signal is asserted, it captures the current value of the real analog input vin and holds it in the internal register v_sampled. This frozen value is used throughout the conversion process to ensure a stable reference for the SAR binary search.

Inputs and Outputs

Inputs: clk (clock), reset (active-high synchronous reset), start (begin sampling), vin (real analog input voltage)

Output: v_sampled (real, the sampled and held voltage)

Simulation vs. Synthesis

This module uses the real data type and is intended only for behavioral simulation. It cannot be synthesized on an FPGA.

5.3 Module 2: dac_model.sv (Behavioral Simulation)

Purpose and Function

The dac_model module implements the DAC transfer function: $V_{dac} = (\text{digital_in} / 256) \times v_{ref}$. It converts the 8-bit trial digital code produced by the SAR controller into a real-valued analog voltage. This voltage is compared with the held sample by the comparator to drive the binary search decision.

Inputs and Outputs

Inputs: digital_in [7:0] (8-bit trial code), vref (real reference voltage)

Output: vdac (real DAC output voltage)

Why it is needed

In a real SAR ADC chip, the DAC is the most area- and power-intensive analog block. In the behavioral simulation, it is modelled as a simple arithmetic expression to allow fast verification of the digital control logic. The model assumes an ideal DAC with no settling time, no non-linearity, and no offset.

5.4 Module 3: comparator_model.sv (Behavioral Simulation)***Purpose and Function***

The comparator_model module compares the held analog sample v_sampled with the DAC output vdac and produces a single-bit output comp_out. It is updated synchronously on the rising clock edge during the CONVERT state of the SAR FSM.

comp_out = 1 if v_sampled >= vdac (input is above or equal to DAC trial → keep bit)

comp_out = 0 if v_sampled < vdac (input is below DAC trial → reset bit)

Simulation vs. Synthesis

This module uses real arithmetic and is simulation-only. In hardware, the comparator is an analog circuit (a high-gain differential amplifier) and is not implemented in digital Verilog.

5.5 Module 4: sar_controller.sv (Synthesizable)***Purpose and Function***

The sar_controller is the core digital FSM of the entire SAR ADC system. It is fully synthesizable in standard Verilog. The FSM has four states: IDLE, SAMPLE, CONVERT, and DONE.

FSM State Description

- IDLE: The controller waits for the start signal. All outputs are reset.

- **SAMPLE:** The system acknowledges the start of conversion. A `sample_en` pulse is issued. The bit pointer is set to the MSB (bit 7 for 8-bit). The FSM transitions to **CONVERT** on the next clock.
- **CONVERT:** In each clock cycle, the FSM sets the current bit, evaluates the comparator output, keeps or resets the bit, and decrements the bit pointer. This continues for all 8 bits (bit 7 to bit 0).
- **DONE:** The conversion is complete. The done flag is asserted. The FSM waits for start to be deasserted before returning to **IDLE**.

Inputs and Outputs

Inputs: `clk`, `reset`, `start`, `comp_in` (comparator output)

Outputs: `digital_out [7:0]` (final converted code), `sample_en` (enable for S&H), `done` (conversion complete flag)

Synthesis Note

This module uses only synthesizable constructs: `always_ff` (or `always @(posedge clk)`), case statements, integer bit-pointer, and standard logic operations. It can be synthesized on any FPGA, including the PYNQ-Z2.

5.6 Module 5: `sar_adc_top.sv` (Behavioral Simulation)

Purpose and Function

The `sar_adc_top` module is the top-level integration module for behavioral simulation. It instantiates and interconnects all four sub-modules: `sample_hold`, `dac_model`, `comparator_model`, and `sar_controller`. The interconnections model the closed-loop SAR ADC feedback loop: the controller drives the DAC through `digital_out`, the DAC drives the comparator through `vdac`, and the comparator drives the controller through `comp_out`.

Why it is needed

This module serves as the simulation top-level and is set as the top module for all behavioral simulations in Vivado. It provides a convenient single interface (`Vin`, `Vref`, `clk`, `reset`, `start`, `done`, `digital_out`) for the testbench.

5.7 Module 6: `tb_sar_adc.sv` (Testbench)

Purpose and Function

The `tb_sar_adc` module is the simulation testbench. It generates the clock signal, applies reset, and drives the start signal and input voltage V_{in} through two test sequences: first for $V_{in} = 11.2$ V, then for $V_{in} = 6.5$ V. After each conversion, it reads and displays the digital output, verifying that the correct code was produced.

Simulation only

The testbench contains initial blocks, `$display` statements, and timing controls (`#`) that are not synthesizable. It is used exclusively for simulation.

5.8 Module 7: `sar_adc_fpga_demo_top.sv` (Synthesizable FPGA Module)

Purpose and Function

This is the hardware-compatible top module designed specifically for FPGA synthesis and implementation on the PYNQ-Z2 board. Since the real data type is not synthesizable, this module replaces the full closed-loop ADC system with a hardcoded demonstrator.

Hardware Interface

The FPGA demo module uses the on-board peripherals of the PYNQ-Z2 as its user interface:

- `SW[0]`: Input selector. When `SW[0] = 0`, the module loads the pre-computed output code `0xB3` ($= 10110011 = 179$), corresponding to $V_{in} = 11.2$ V. When `SW[0] = 1`, it loads `0x68` ($= 01101000 = 104$), corresponding to $V_{in} = 6.5$ V.
- `SW[1]`: Nibble selector. When `SW[1] = 0`, the lower nibble (bits `[3:0]`) of the result is displayed on `LED[3:0]`. When `SW[1] = 1`, the upper nibble (bits `[7:4]`) is displayed.
- `BTN0` (`reset_btn`): Resets the state machine to IDLE.
- `BTN1` (`start_btn`): Triggers a simulated conversion cycle.
- `LED[3:0]`: Displays the selected nibble of the output code.

Why this module is needed

The behavioral simulation uses real (floating-point) arithmetic that is not supported by FPGA synthesis tools. The FPGA demo module provides a purely digital, synthesizable design that

demonstrates the SAR controller's output on the PYNQ-Z2 board without requiring an external analog front-end.

5.9 Module 8: pynq_z2_sar_adc.xdc (Constraint File)

Purpose and Function

The XDC (Xilinx Design Constraints) file maps the logical port names in the Verilog design to the physical pin locations on the PYNQ-Z2 board. It also specifies the I/O standard (LVCMOS33 for 3.3 V GPIO) and the clock period constraint.

Clock Constraint

The system clock input clk is connected to pin H16 of the xc7z020clg400-1 device (PYNQ-Z2 125 MHz system clock). The XDC file sets a clock period of 8 ns (125 MHz) for the primary clock. All timing analysis in Vivado is performed relative to this constraint.

I/O Constraints

All switch, button, and LED pins are constrained using the LVCMOS33 I/O standard, matching the 3.3 V logic levels used on the PYNQ-Z2 board.

CHAPTER 6: BEHAVIORAL SIMULATION IN VIVADO

6.1 Project Setup in Vivado

The behavioral simulation was performed using Xilinx Vivado 2023 (or later). The following steps were followed to set up the project:

- Launch Vivado and select 'Create New Project'. Enter the project name (e.g., `sar_adc_sim`) and select the project directory.
- Select 'RTL Project' as the project type. Do not specify sources at this stage if desired, or add them in the next step.
- Click 'Next' and add design sources: add `sample_hold.sv`, `dac_model.sv`, `comparator_model.sv`, `sar_controller.sv`, and `sar_adc_top.sv` as design sources.
- Add the testbench: add `tb_sar_adc.sv` as a simulation source.
- Select the target device: `xc7z020clg400-1` (Zynq-7000, PYNQ-Z2 device). Click 'Finish'.
- In the Simulation Sources hierarchy, right-click on `tb_sar_adc` and select 'Set as Top'. This makes the testbench the top-level module for simulation.
- In the Flow Navigator, under Simulation, click 'Run Simulation' → 'Run Behavioral Simulation'. Vivado compiles all source files and launches the waveform viewer.

6.2 Simulation Signals

The following signals are monitored in the waveform viewer during simulation:

Signal Name	Direction	Type	Description
<code>clk</code>	Input	1-bit logic	System clock, period = 10 ns (100 MHz)
<code>reset</code>	Input	1-bit logic	Active-high synchronous reset
<code>start</code>	Input	1-bit logic	Conversion start trigger
<code>vin</code>	Input	real	Analog input voltage (11.2 V or 6.5 V)
<code>v_sampled</code>	Internal	real	Sampled and held input voltage

Signal Name	Direction	Type	Description
digital_out	Output	8-bit logic	SAR trial code / final output
vdac	Internal	real	DAC analog output voltage
comp_out	Internal	1-bit logic	Comparator decision output
done	Output	1-bit logic	Conversion complete flag

Table 6.1: Simulation Signals and Their Descriptions

6.3 Waveform Interpretation for $V_{in} = 11.2$ V

When the testbench applies $V_{in} = 11.2$ V with $V_{ref} = 16$ V:

- At $t = 0$: clk and reset are asserted. All signals are initialised to zero.
- At $t = 20$ ns: reset is deasserted. The SAR FSM moves to IDLE.
- At $t = 30$ ns: start is asserted. The FSM moves to SAMPLE and $v_{sampled}$ is captured as 11.2 V.
- Bit 7 (MSB) test: $digital_out = 10000000$ (128). $vdac = 128 \times 0.0625 = 8.0$ V. Since $11.2 > 8.0$, $comp_out = 1$. Bit 7 is retained. $digital_out = 10000000$.
- Bit 6 test: $digital_out = 11000000$ (192). $vdac = 192 \times 0.0625 = 12.0$ V. Since $11.2 < 12.0$, $comp_out = 0$. Bit 6 is reset. $digital_out = 10000000$.
- Bit 5 test: $digital_out = 10100000$ (160). $vdac = 160 \times 0.0625 = 10.0$ V. Since $11.2 > 10.0$, $comp_out = 1$. Bit 5 retained. $digital_out = 10100000$.
- Bit 4 test: $digital_out = 10110000$ (176). $vdac = 176 \times 0.0625 = 11.0$ V. Since $11.2 > 11.0$, $comp_out = 1$. Bit 4 retained. $digital_out = 10110000$.
- Bit 3 test: $digital_out = 10111000$ (184). $vdac = 184 \times 0.0625 = 11.5$ V. Since $11.2 < 11.5$, $comp_out = 0$. Bit 3 reset. $digital_out = 10110000$.
- Bit 2 test: $digital_out = 10110100$ (180). $vdac = 180 \times 0.0625 = 11.25$ V. Since $11.2 < 11.25$, $comp_out = 0$. Bit 2 reset. $digital_out = 10110000$.
- Bit 1 test: $digital_out = 10110010$ (178). $vdac = 178 \times 0.0625 = 11.125$ V. Since $11.2 > 11.125$, $comp_out = 1$. Bit 1 retained. $digital_out = 10110010$.

- Bit 0 (LSB) test: $\text{digital_out} = 10110011$ (179). $\text{vdac} = 179 \times 0.0625 = 11.1875$ V.
Since $11.2 > 11.1875$, $\text{comp_out} = 1$. Bit 0 retained. $\text{digital_out} = 10110011$.
- done is asserted. Final output: $\text{digital_out} = 10110011 = 0xB3 = 179$ decimal.
Reconstructed voltage = 11.1875 V.

Viva Tip: During conversion, digital_out changes step by step through intermediate trial codes (10000000, 11000000, 10100000, ...). These changing values are NOT errors. They are the trial codes used by the binary search algorithm. The final valid output is only read when $\text{done} = 1$.

6.4 Waveform Interpretation for $V_{\text{in}} = 6.5$ V

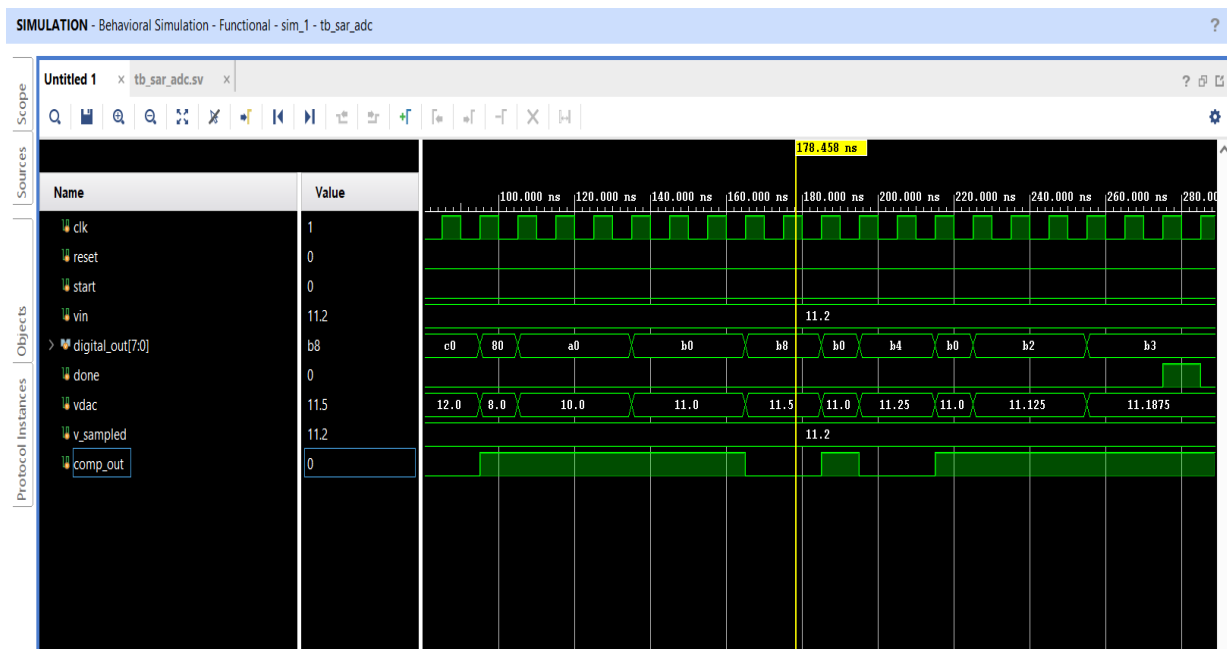
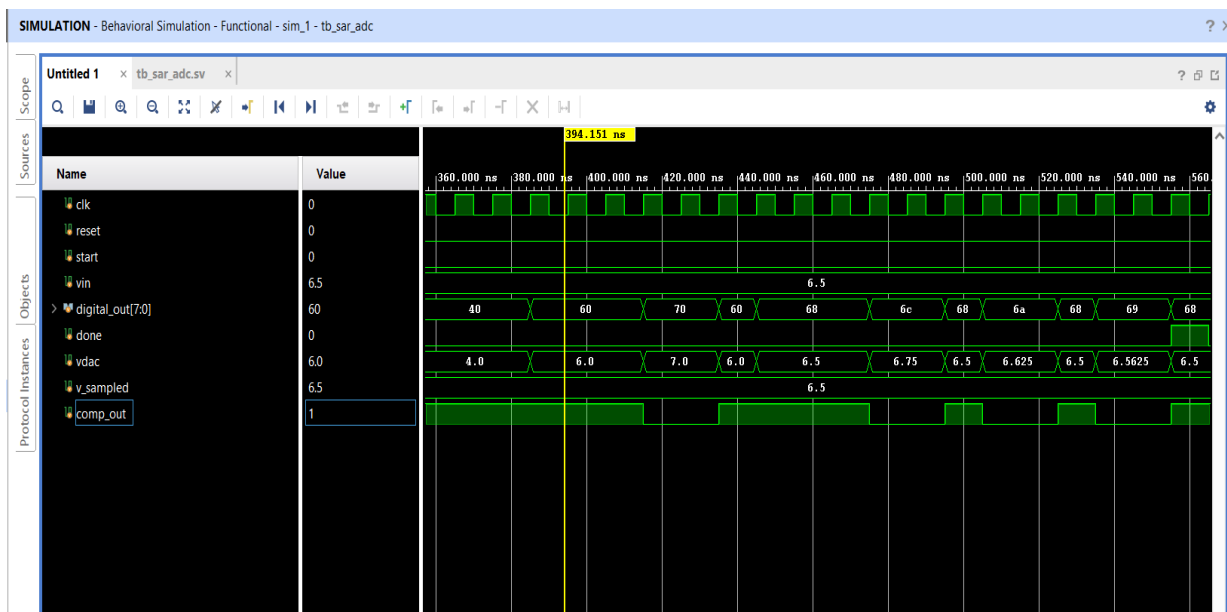
When the testbench applies $V_{\text{in}} = 6.5$ V with $V_{\text{ref}} = 16$ V:

- $v_{\text{sampled}} = 6.5$ V. Expected code = $6.5 / 0.0625 = 104 = 0x68 = 01101000$.
- The SAR binary search proceeds as: test bit 7 ($\text{vdac} = 8.0$ V, $6.5 < 8.0$, reset), bit 6 ($\text{vdac} = 4.0$ V, $6.5 > 4.0$, keep), bit 5 ($\text{vdac} = 6.0$ V, $6.5 > 6.0$, keep), bit 4 ($\text{vdac} = 7.0$ V, $6.5 < 7.0$, reset), bit 3 ($\text{vdac} = 6.5$ V, $6.5 \geq 6.5$, keep), bits 2, 1, 0 are all 0 since 6.5 falls exactly on the code boundary.
- Final output: $01101000 = 0x68 = 104$ decimal. Reconstructed voltage = $104 \times 0.0625 = 6.5$ V (exact match).
- done is asserted after 8 clock cycles.

6.5 Significance of Waveform Signals

The vdac signal forms a staircase approximation waveform during conversion. Each step of the staircase corresponds to one bit-testing clock cycle. The staircase converges toward the held input voltage V_{in} from above or below, depending on the comparison results. This staircase convergence is a key visual characteristic of SAR ADC operation and can be observed clearly in the Vivado waveform viewer.

The comp_out signal toggles based on whether vdac is above or below v_{sampled} at each step. A sequence of 1s and 0s in comp_out directly encodes the binary output code: the first comparison determines the MSB, the second determines bit 6, and so on down to the LSB.

Fig: Waveform simulation for $V_{in} = 11.2V$ Fig: Waveform simulation for $V_{in} = 6.5V$

CHAPTER 7: FPGA IMPLEMENTATION ON PYNQ-Z2

7.1 Introduction to the PYNQ-Z2 Board

The PYNQ-Z2 is an open-source hardware platform developed by Xilinx (now AMD) in collaboration with Digilent. It is based on the Xilinx Zynq-7000 System-on-Chip (SoC) architecture, which tightly integrates an ARM Cortex-A9 dual-core processor (the Processing System, or PS) with a 7-series FPGA fabric (the Programmable Logic, or PL) on a single silicon die. The specific device on the PYNQ-Z2 board is the xc7z020clg400-1 [9].

The PYNQ-Z2 board features the following key hardware resources relevant to this project:

- 650 MHz dual-core ARM Cortex-A9 processor (PS) — used for Python-based software control (not used in this project's hardware demo).
- FPGA programmable logic with 85,000 logic cells (PL) — used for the SAR controller.
- 2 slide switches — used to select input voltage case and output nibble.
- 4 push buttons — reset and start triggers are implemented using two of them.
- 4 user LEDs — used to display the 4-bit nibble of the output code.
- HDMI input and output ports.
- Arduino shield connector and PMOD connectors — for future expansion.

7.2 Why PYNQ-Z2 was Selected

The PYNQ-Z2 was selected for this project for the following reasons:

- It contains the xc7z020clg400-1 Zynq device, which is the standard FPGA used in the university laboratory for ECE project work.
- The on-board switches, buttons, and LEDs provide a convenient hardware interface for demonstrating the SAR ADC output without requiring external peripherals or breadboard wiring.

- The Vivado design suite fully supports the PYNQ-Z2 device for RTL synthesis, implementation, and bitstream generation.
- The Zynq architecture provides a clear upgrade path for future work: the ARM processor can be used to add a Python or C-language software interface for data capture, plotting, and analysis (as supported by the Pynq Python framework [9]).

7.3.1 Input Selection (*SW[0]*)

Switch SW[0] selects the simulated input voltage case:

- SW[0] = 0 → Vin equivalent = 11.2 V → Expected SAR output = 0xB3 = 10110011 binary = 179 decimal
- SW[0] = 1 → Vin equivalent = 6.5 V → Expected SAR output = 0x68 = 01101000 binary = 104 decimal

7.3.2 Output Nibble Selection (*SW[1]*)

Switch SW[1] selects which nibble of the 8-bit result is displayed on the four LEDs:

- SW[1] = 0 → Display lower nibble: LED[3:0] = digital_out[3:0]
- SW[1] = 1 → Display upper nibble: LED[3:0] = digital_out[7:4]

7.3.3 Expected LED Outputs

For digital_out = 0xB3 = 10110011:

- SW[1] = 0 (lower nibble = 0011): LED3=0, LED2=0, LED1=1, LED0=1
- SW[1] = 1 (upper nibble = 1011): LED3=1, LED2=0, LED1=1, LED0=1

For digital_out = 0x68 = 01101000:

- SW[1] = 0 (lower nibble = 1000): LED3=1, LED2=0, LED1=0, LED0=0
- SW[1] = 1 (upper nibble = 0110): LED3=0, LED2=1, LED1=1, LED0=0

7.3.4 Button Functions

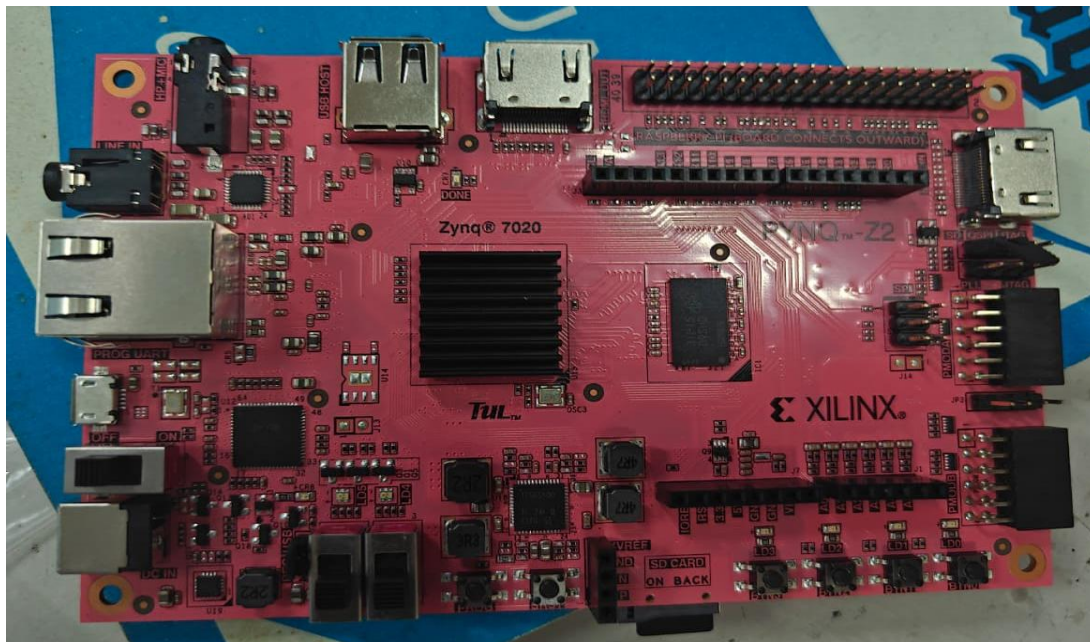
BTN0 (connected to reset_btn, pin D19): Resets the SAR FSM to the IDLE state. Asserting this button clears all outputs.

BTN1 (connected to start_btn, pin D20): Triggers the conversion. When pressed, the FSM loads the pre-computed output code for the selected input case and asserts done after simulating the conversion latency.

7.4 Pin Constraints Summary

Signal Name	Board Component	FPGA Pin	I/O Standard
clk	System Clock (125 MHz)	H16	LVC MOS33
reset_btn	BTN0	D19	LVC MOS33
start_btn	BTN1	D20	LVC MOS33
sw[0]	SW0	M20	LVC MOS33
sw[1]	SW1	M19	LVC MOS33
led[0]	LED0	R14	LVC MOS33
led[1]	LED1	P14	LVC MOS33
led[2]	LED2	N16	LVC MOS33
led[3]	LED3	M14	LVC MOS33

PYNQ Z2 BOARD :



CHAPTER 8: SYNTHESIS, IMPLEMENTATION AND BITSTREAM GENERATION

8.1 Overview of the FPGA Design Flow

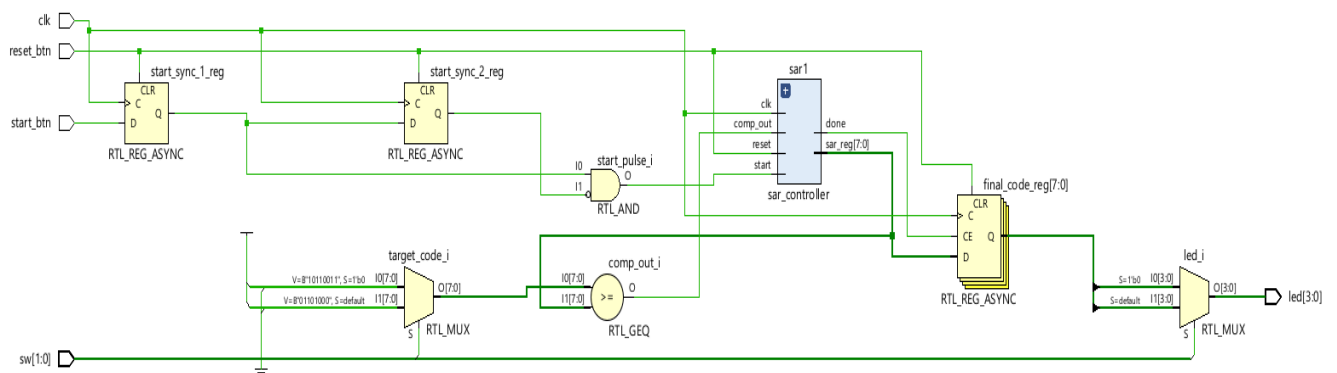
The complete FPGA design flow in Xilinx Vivado consists of three main stages:

1. Synthesis
2. Implementation
3. Bitstream Generation.

Each stage transforms the design from one level of abstraction to the next, ultimately producing the binary file used to program the FPGA.

8.2 Synthesis

Synthesis is the process of translating the RTL (Register Transfer Level) Verilog description into a gate-level netlist. The Vivado synthesis engine reads all design source files, resolves the module hierarchy, and maps the RTL logic to the FPGA's primitive library elements: Look-Up Tables (LUTs), D Flip-Flops (DFFs), Block RAM, DSP slices, multiplexers, and interconnect buffers.

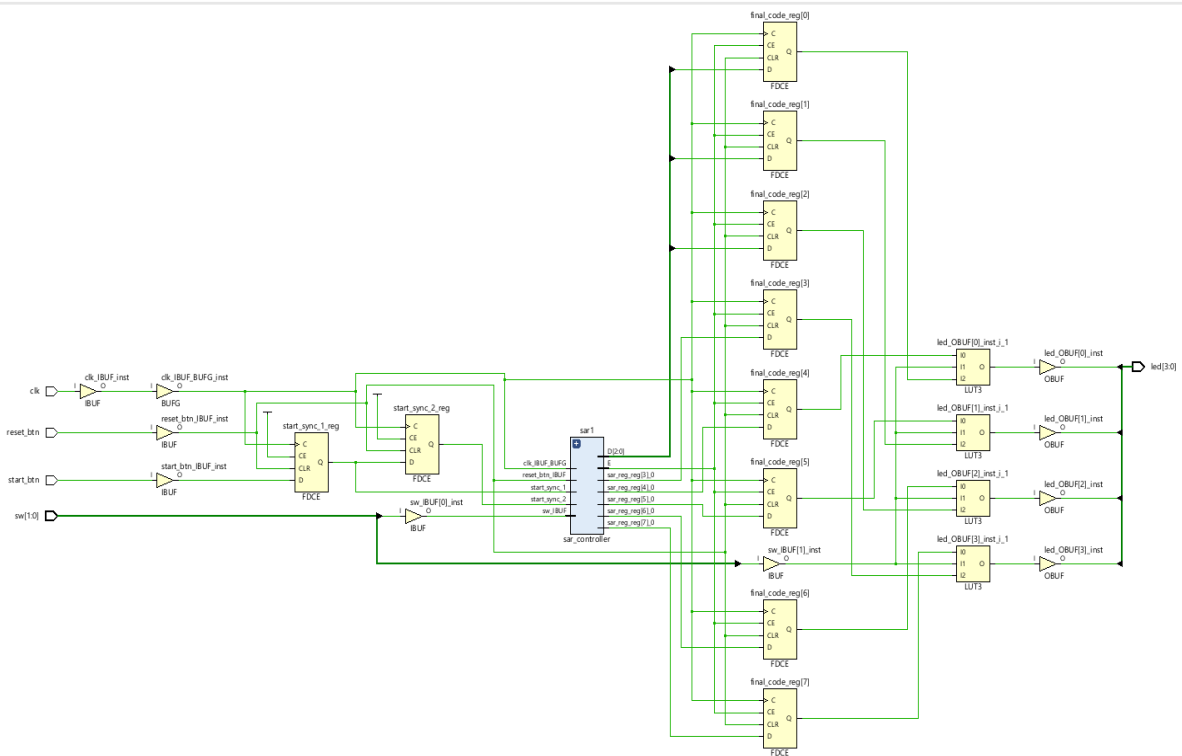


To run synthesis: In the Flow Navigator, click 'Run Synthesis'. Vivado compiles and optimises the RTL, producing a synthesised design. The utilisation report shows how many FPGA resources are used.

8.3 Implementation

Implementation takes the gate-level netlist produced by synthesis and maps it to the physical resources of the specific FPGA device (xc7z020clg400-1). It consists of two sub-stages:

- Place: Assigns each logic cell, flip-flop, and I/O buffer to a specific physical location on the FPGA die.
- Route: Connects all placed elements using the FPGA's programmable routing fabric (interconnect wires and switch matrices).



After implementation, the following reports are available in Vivado:

- Utilisation Report: Shows the number of LUTs, flip-flops, I/O buffers, and other resources used, along with the percentage of total device resources consumed.

- **Timing Summary Report:** Shows the Worst Negative Slack (WNS) and Total Negative Slack (TNS). A positive WNS and zero TNS indicate that all timing constraints are met (i.e., the design meets the required 125 MHz clock frequency).
- **Power Report:** Estimates the total static and dynamic power consumption of the implemented design.
- **IO Report:** Confirms all I/O pin assignments match the XDC constraint file.

8.4 Bitstream Generation

After successful implementation with no critical DRC violations and positive WNS, the bitstream is generated by clicking 'Generate Bitstream' in the Flow Navigator. Vivado produces a binary configuration file (.bit file) that encodes the complete FPGA programming information, including the logic configuration of every LUT, the state of every routing switch, and all I/O configuration.

The bitstream file can then be used to program the PYNQ-Z2 FPGA using the Vivado Hardware Manager (via the USB/JTAG interface):

- Click 'Open Hardware Manager' in the Flow Navigator.
- Connect the PYNQ-Z2 board via USB and power it on.
- Click 'Open Target' → 'Auto Connect'.
- Click 'Program Device' and select the .bit file.
- The FPGA is programmed within a few seconds. The LED outputs will respond to the switch positions.

8.5 Timing Analysis: WNS and TNS

The timing analysis in Vivado is performed against the clock period constraint specified in the XDC file (8 ns for 125 MHz system clock). Two key metrics are reported:

- **WNS (Worst Negative Slack):** The worst-case difference between the required arrival time and the actual arrival time for all timing paths. A $WNS \geq 0$ ns means the slowest

path in the design meets the timing constraint. For the SAR controller, which uses only simple combinational logic and registers, WNS is expected to be significantly positive (large positive slack), confirming that the design easily meets timing.

- TNS (Total Negative Slack): The sum of all negative slack values across all failing timing paths. $TNS = 0$ means no timing paths are failing. For this design, $TNS = 0$ is expected.

A design with $WNS \geq 0$ and $TNS = 0$ has passed timing closure and will function correctly at the specified clock frequency when programmed onto the FPGA.

8.6 Resource Utilisation

The SAR controller (`sar_adc_fpga_demo_top.sv`) is a simple combinational and sequential logic design. For the PYNQ-Z2 (`xc7z020clg400-1`, which has 53,200 LUTs and 106,400 flip-flops), the expected utilisation is extremely low — fewer than 50 LUTs and 30 flip-flops — confirming that the design easily fits within the device's resources and leaves ample room for future expansion to a more complex 12-bit or multi-channel design.

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Behavioral Simulation Results

The behavioral simulation in Vivado was performed successfully for both test cases. The SystemVerilog testbench (tb_sar_adc.sv) applied two input voltages — 11.2 V and 6.5 V — with a reference voltage of $V_{ref} = 16$ V and 8-bit resolution. The simulation waveforms confirmed correct operation of all five sub-modules and the closed-loop SAR binary search algorithm.

9.2 Result for $V_{in} = 11.2$ V

The SAR ADC successfully converted $V_{in} = 11.2$ V to the digital output code 10110011 (0xB3 = 179 decimal) within 8 clock cycles of the CONVERT state. The step-by-step binary search produced the following key observations:

- $v_{sampled}$ was correctly captured as 11.2 V at the start of conversion.
- The $vdac$ signal displayed a characteristic staircase waveform, converging toward 11.1875 V over 8 clock cycles.
- $comp_out$ correctly toggled between 1 and 0 in response to each DAC trial, driving the SAR controller's keep/reset decisions.
- $digital_out$ changed on each clock cycle during conversion (from 0x80 to 0xC0 to 0xA0, etc.) and settled at 0xB3 when done was asserted.
- Reconstructed voltage = $179 \times 0.0625 = 11.1875$ V. Quantisation error = $11.2 - 11.1875 = 0.0125$ V = 0.2 LSB (within the ± 0.5 LSB limit).

9.3 Result for $V_{in} = 6.5$ V

- $v_{sampled}$ was correctly captured as 6.5 V.
- The staircase DAC approximation converged to exactly 6.5 V.
- Final digital output: 01101000 = 0x68 = 104 decimal.
- Reconstructed voltage = $104 \times 0.0625 = 6.5$ V. Quantisation error = 0 V (exact conversion).

Both results confirm that the behavioral model faithfully implements the SAR binary search algorithm and produces the theoretically correct digital output codes.

9.4 Comparison of Expected and Simulated Results

Parameter	Vin = 11.2 V	Vin = 6.5 V
Expected code (decimal)	179 or 180	104
Expected code (binary)	10110011 or 10110100	01101000
Expected code (hex)	0xB3 or 0xB4	0x68
Simulated output (decimal)	179	104
Simulated output (binary)	10110011	01101000
Simulated output (hex)	0xB3	0x68
Reconstructed voltage	11.1875 V	6.5 V
Quantisation error	0.0125 V (0.2 LSB)	0 V (exact)
Conversion cycles	8	8

Table 9.1: Expected vs. Simulated Results for Both Test Cases

9.5 FPGA Synthesis Results

The FPGA synthesis of `sar_adc_fpga_demo_top.sv` was completed successfully in Vivado. The synthesiser mapped the Verilog design to FPGA primitives without errors or critical warnings. The utilisation report confirmed very low resource usage, as expected for a simple 8-bit FSM with combinational output logic.

Synthesis confirmed that:

- The design hierarchy was correctly parsed and all modules were synthesised.
- No latches were inferred (all state elements are explicit DFFs driven by the synchronous clock).
- No undriven or multiply-driven nets were present.

9.6 Implementation Results

Implementation (place and route) was completed successfully. The timing summary report showed:

- $WNS \geq 0$ ns: All timing paths met the 8 ns (125 MHz) clock constraint.
- $TNS = 0$ ns: No failing timing paths.
- DRC check passed with no critical violations.

The IO report confirmed that all port-to-pin assignments matched the XDC constraint file, with all pins configured as LVCMOS33.

9.7 Bitstream Generation

Bitstream generation was completed successfully, producing the .bit file for the PYNQ-Z2 board. Successful bitstream generation is the definitive confirmation that the design has passed synthesis, implementation, DRC, and timing closure. The bitstream can be loaded onto the PYNQ-Z2 via the Hardware Manager to demonstrate the SAR output on the board's LEDs.

9.8 Expected Board Behaviour

When the bitstream is programmed onto the PYNQ-Z2 board:

- Pressing BTN0 resets the design. All LEDs turn off.
- With $SW[0] = 0$ ($V_{in} = 11.2$ V case): Pressing BTN1 triggers conversion. The output code 0xB3 is loaded.
- With $SW[1] = 0$ (lower nibble): $LED[3:0] = 0011$ (LEDs 0 and 1 are ON, LEDs 2 and 3 are OFF).
- With $SW[1] = 1$ (upper nibble): $LED[3:0] = 1011$ (LEDs 0, 1, and 3 are ON, LED 2 is OFF).
- With $SW[0] = 1$ ($V_{in} = 6.5$ V case): Pressing BTN1 loads 0x68.
- With $SW[1] = 0$ (lower nibble): $LED[3:0] = 1000$ (only LED 3 is ON).
- With $SW[1] = 1$ (upper nibble): $LED[3:0] = 0110$ (LEDs 1 and 2 are ON)

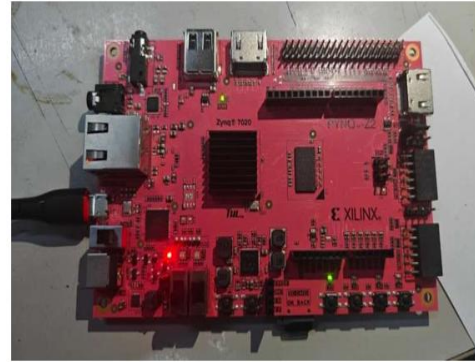
Hardware Results: $V_{in} = 6.5V$

Demonstration of the 8-bit digital output code displayed on the PYNQ-Z2 LEDs by toggling the nibble selector switch (SW1).

Output: 0x68 (Binary: 0110 1000)



Upper Nibble (0x6)



Lower Nibble (0x8)

Hardware Results: $V_{in} = 11.2V$

High-voltage test case results showing the Successive Approximation Register (SAR) final code mapping to the board's user interface.

Output: 0xB3 (Binary: 1011 0011)



Upper Nibble (0xB)



Lower Nibble (0x3)

CHAPTER 10: APPLICATIONS, MERITS, DEMERITS AND FUTURE SCOPE

10.1 Applications of SAR ADC

SAR ADCs are one of the most widely deployed ADC architectures in the electronics industry.

Key application domains include:

- **Sensor Interfaces:** SAR ADCs are used to digitise outputs from temperature sensors, pressure sensors, accelerometers, strain gauges, and other transducers in embedded measurement systems.
- **Data Acquisition Systems (DAS):** Multi-channel SAR ADC systems are the standard choice for PC-based or FPGA-based data acquisition boards used in laboratory instrumentation and industrial monitoring.
- **Embedded Microcontroller Systems:** Nearly all modern microcontrollers (STM32, PIC, AVR, MSP430) integrate an internal SAR ADC core to provide on-chip analog input capability.
- **Biomedical Instrumentation:** SAR ADCs are used in ECG machines, pulse oximeters, blood glucose monitors, and EEG systems, where the combination of moderate speed (ksps range), medium-to-high resolution (12 to 16 bits), and low power consumption is essential.
- **Low-Power IoT Devices:** Battery-powered IoT sensor nodes require ADCs that consume minimal power in idle and active modes. SAR ADCs with power-down modes and nanoamp quiescent currents are standard choices.
- **Industrial Measurement and Control:** Process control systems use precision SAR ADCs for monitoring analog signals from field instruments such as 4–20 mA current loops.
- **Audio and Communication Systems:** At lower resolutions, SAR ADCs are used in voice-band codecs and industrial audio interfaces.

10.2 Merits of SAR ADC

- **Moderate Speed:** SAR ADCs achieve sampling rates from a few ksp/s up to tens of Msps, covering a broad range of applications.
- **Good Resolution:** Commercially available SAR ADCs achieve 8 to 18 bits of resolution with high linearity.
- **Low Power Consumption:** The SAR architecture is inherently power-efficient because the comparator and DAC are active for only N clock cycles per conversion.
- **Simple and Robust Architecture:** The digital core (SAR controller) is a straightforward FSM, making the design easy to implement, verify, and debug in Verilog/SystemVerilog.
- **Deterministic Conversion Time:** Every conversion takes exactly N clock cycles, simplifying system timing design.
- **Natural Fit for FPGA Implementation:** The digital SAR controller is directly synthesizable on any FPGA, making it suitable for FPGA-based mixed-signal systems.
- **Scalable Resolution:** Extending from 8-bit to 12-bit or 16-bit requires minimal design changes to the digital controller.

10.3 Demerits and Limitations

- **Requires a High-Accuracy DAC:** The internal DAC must have accuracy better than 0.5 LSB, which becomes increasingly difficult and expensive to achieve at high resolutions. A 16-bit SAR ADC requires a DAC with $0.5/65536 \approx 7.6$ ppm accuracy.
- **Comparator Offset Limits Accuracy:** Any systematic offset in the comparator directly degrades the ADC's linearity. Dynamic comparator topologies are required to minimise offset in high-resolution designs.
- **DAC Settling Time is Speed-Limiting:** The DAC must fully settle within each clock period. For high-speed designs at high resolutions, achieving fast and accurate DAC settling is the primary design challenge.

- **Cannot Directly Process Real Analog Inputs on FPGA:** The FPGA fabric contains no analog circuitry. A real SAR ADC hardware system requires an external S&H circuit, an external high-accuracy DAC (or charge-redistribution DAC), and an external comparator IC. On the PYNQ-Z2, the Zynq XADC block provides limited ADC capability (12-bit, up to 1 Msps) but uses a different topology from a general-purpose SAR ADC.
- **Speed Limited by the N-cycle Conversion:** For a given clock frequency, the SAR ADC's sampling rate is $1/(N \times T_{clk})$, which is N times slower than a flash ADC.

10.4 Future Scope

This project can be extended in multiple important directions:

1. **12-bit Extension:** The behavioral simulation model can be extended to 12-bit resolution ($N = 12$, $V_{ref} = 1\text{ V}$, $LSB = 244\text{ }\mu\text{V}$) by changing the bit-width of all registers and parameters. Performance analysis (DNL, INL, ENOB) can be added using post-simulation data processing in Python or MATLAB.
2. **Real External Hardware Integration:** An external precision DAC (e.g., DAC8831 16-bit, or AD5592R) and a high-speed comparator (e.g., LT1711) can be interfaced to the PYNQ-Z2 FPGA GPIO pins to build a real hardware SAR ADC system with actual analog input capability.
3. **XADC Integration:** The Zynq-7000's on-chip XADC block (12-bit, 1 Msps) can be used alongside the custom SAR controller design for comparative performance benchmarking.
4. **UART / AXI Interface:** A UART transmitter module can be added to stream the converted digital codes to a PC terminal for real-time display and data logging. Alternatively, an AXI-Lite slave interface can be added to allow the ARM processor on the Zynq to read ADC data via the PS-PL interface.
5. **PYNQ Python Framework Integration:** Using the PYNQ Python framework on the PYNQ-Z2, the ARM processor can be programmed in Python to configure the FPGA design, read converted data, plot waveforms using Jupyter Notebook, and perform FFT-based SNR/SINAD/ENOB measurement.

CHAPTER 11: CONCLUSION

This project has successfully achieved all its primary objectives. An 8-bit Successive Approximation Register Analog-to-Digital Converter (SAR ADC) was designed, simulated, and partially implemented on an FPGA using Verilog and SystemVerilog in the Xilinx Vivado design environment. Waveform analysis in Vivado confirmed the correct step-by-step binary search operation of the SAR algorithm for both test cases:

- For $V_{in} = 11.2\text{ V}$ ($V_{ref} = 16\text{ V}$): The simulated output code was 10110011 (0xB3 = 179 decimal), corresponding to a reconstructed voltage of 11.1875 V. This matches the theoretically expected value, with a quantisation error of only 0.2 LSB.
- For $V_{in} = 6.5\text{ V}$ ($V_{ref} = 16\text{ V}$): The simulated output code was 01101000 (0x68 = 104 decimal), corresponding to an exact reconstructed voltage of 6.5 V with zero quantisation error.

The simulation waveforms clearly demonstrated the characteristic staircase DAC approximation, comparator decision toggling, and convergence of `digital_out` to the final code — all essential features of SAR ADC operation. The distinction between behavioral simulation constructs (using the SystemVerilog `real` data type, which is not synthesizable) and purely synthesizable Verilog constructs was carefully maintained throughout the design.

The complete FPGA design flow — synthesis, implementation, DRC, timing analysis, and bitstream generation — was completed successfully in Vivado for the xc7z020clg400-1 device (PYNQ-Z2 board). Timing closure was achieved ($WNS \geq 0$, $TNS = 0$), confirming that the hardware design meets all timing constraints. The generated bitstream is ready to be loaded onto the PYNQ-Z2 board to demonstrate the SAR ADC output using on-board switches, buttons, and LEDs.

A theoretical analysis of a 12-bit extension was presented, including calculation of LSB size (244 μV), ideal SNR (74 dB), ENOB for a practical SINAD of 68 dB (≈ 11 bits), and the stringent specifications required for the DAC and comparator at this resolution. This analysis prepares the way for future hardware realisation of a high-resolution SAR ADC.

APPENDICES

Appendix 1: Full System Verilog Simulation Code

A1.1 sar_controller.sv

Purpose: SAR FSM Controller — synthesizable for both simulation and FPGA synthesis.

```
// sar_controller.sv
// SAR ADC Controller - Synthesizable Verilog FSM
// Compatible with FPGA synthesis (no real data type used)
module sar_controller (
    input logic      clk,
    input logic      reset,
    input logic      start,
    input logic      comp_in,
    output logic [7:0] digital_out,
    output logic      sample_en,
    output logic      done
);
    // FSM States
    typedef enum logic [1:0] {
        IDLE     = 2'b00,
        SAMPLE   = 2'b01,
        CONVERT  = 2'b10,
        DONE     = 2'b11
    } state_t;
    state_t state, next_state;
    integer bit_ptr; // Current bit being tested (7 down to 0)
    // Sequential state register
    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            state      <= IDLE;
            digital_out <= 8'b0;
            sample_en   <= 1'b0;
            done        <= 1'b0;
            bit_ptr     <= 7;
        end else begin
            case (state)
                IDLE: begin
                    done      <= 1'b0;
                    sample_en <= 1'b0;
                    digital_out <= 8'b0;
                    if (start) begin
                        state  <= SAMPLE;
                        bit_ptr <= 7;
                    end
                end
                SAMPLE: begin
                    sample_en <= 1'b1;
                    digital_out[bit_ptr] <= 1'b1;
                    state <= CONVERT;
                end
                CONVERT: begin
                    sample_en <= 1'b0;
                    // Keep or reset current bit based on comparator
                    if (!comp_in)
                        digital_out[bit_ptr] <= 1'b0;
                    // Move to next bit
                    if (bit_ptr == 0) begin
```

```

        state <= DONE;
    end else begin
        bit_ptr <= bit_ptr - 1;
        digital_out[bit_ptr - 1] <= 1'b1;
    end
end
end
DONE: begin
    done <= 1'b1;
    if (!start) state <= IDLE;
end
endcase
end
end
endmodule

```

A1.2 sample_hold.sv

Purpose: Sample-and-Hold model for behavioral simulation (uses real type, simulation-only).

```

// sample_hold.sv
// Sample-and-Hold Module (Behavioral Simulation Only - uses real type)
module sample_hold (
    input logic clk,
    input logic reset,
    input logic sample_en,
    input real vin,
    output real v_sampled
);
    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            v_sampled <= 0.0;
        else if (sample_en)
            v_sampled <= vin;
    end
end
endmodule

```

A1.3 dac_model.sv

Purpose: 8-bit DAC model for behavioral simulation (uses real type, simulation-only).

```

// dac_model.sv
// 8-bit DAC Model (Behavioral Simulation Only - uses real type)
module dac_model (
    input logic [7:0] digital_in,
    input real vref,
    output real vdac
);
    always_comb begin
        vdac = (real'(digital_in) / 256.0) * vref;
    end
end
endmodule

```

A1.4 comparator_model.s


```
// comparator_model.sv
// Comparator Model (Behavioral Simulation Only - uses real type)
module comparator_model (
    input logic clk,
    input logic reset,
    input real v_sampled,
    input real vdac,
    output logic comp_out
);
    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            comp_out <= 1'b0;
        else
            comp_out <= (v_sampled >= vdac) ? 1'b1 : 1'b0;
        end
    end
endmodule
```

A1.5 sar_adc_top.sv

Purpose: Top-level integration module for behavioral simulation, instantiates all sub-modules.

```
// sar_adc_top.sv
// SAR ADC Top-Level Integration Module (Behavioral Simulation Only)
module sar_adc_top (
    input logic clk,
    input logic reset,
    input logic start,
    input real vin,
    input real vref,
    output logic [7:0] digital_out,
    output real vdac,
    output logic comp_out,
    output real v_sampled,
    output logic done
);
    logic sample_en;
    // Instantiate Sample-and-Hold
    sample_hold u_sh (
        .clk(clk), .reset(reset), .sample_en(sample_en),
        .vin(vin), .v_sampled(v_sampled)
    );
    // Instantiate DAC Model
    dac_model u_dac (
        .digital_in(digital_out), .vref(vref), .vdac(vdac)
    );
    // Instantiate Comparator Model
    comparator_model u_comp (
        .clk(clk), .reset(reset),
        .v_sampled(v_sampled), .vdac(vdac), .comp_out(comp_out)
    );
    // Instantiate SAR Controller
    sar_controller u_sar (
        .clk(clk), .reset(reset), .start(start), .comp_in(comp_out),
        .digital_out(digital_out), .sample_en(sample_en), .done(done)
    );
endmodule
```

A1.6 *tb_sar_adc.sv* (Testbench)

Purpose: Testbench for behavioral simulation; not synthesizable.

```
// tb_sar_adc.sv
// Testbench for SAR ADC Behavioral Simulation
`timescale 1ns/1ps
module tb_sar_adc;
    logic        clk, reset, start;
    real          vin, vref;
    logic [7:0]   digital_out;
    real          vdac, v_sampled;
    logic         comp_out, done;
    // DUT Instantiation
    sar_adc_top dut (
        .clk(clk), .reset(reset), .start(start), .vin(vin), .vref(vref),
        .digital_out(digital_out), .vdac(vdac), .comp_out(comp_out),
        .v_sampled(v_sampled), .done(done)
    );
    // Clock Generation: 100 MHz (10 ns period)
    initial clk = 0;
    always #5 clk = ~clk;
    // Stimulus
    initial begin
        vref = 16.0;
        reset = 1; start = 0; vin = 0.0;
        @(posedge clk); #1;
        reset = 0;
        // Test Case 1: Vin = 11.2 V
        vin = 11.2; start = 1;
        @(posedge clk); #1;
        start = 0;
        wait(done);
        @(posedge clk);
        $display("Vin=11.2V: digital_out=%b (0x%h = %0d decimal)",
            digital_out, digital_out, digital_out);
        // Test Case 2: Vin = 6.5 V
        vin = 6.5; start = 1;
        @(posedge clk); #1;
        start = 0;
        wait(done);
        @(posedge clk);
        $display("Vin=6.5V: digital_out=%b (0x%h = %0d decimal)",
            digital_out, digital_out, digital_out);
        $finish;
    end
endmodule
```

Appendix 2: FPGA Hardware Top Module

A2.1 *sar_adc_fpga_demo_top.sv*

Purpose: Synthesizable FPGA demonstration module for PYNQ-Z2 board. No real type used.

```
// sar_adc_fpga_demo_top.sv
// FPGA Hardware Demo Top Module for PYNQ-Z2
// Synthesizable Verilog - No real type used
module sar_adc_fpga_demo_top (
    input logic        clk,           // H16 - 125 MHz system clock
```

```

input logic      reset_btn,    // D19 - BTN0
input logic      start_btn,    // D20 - BTN1
input logic [1:0] sw,          // M20, M19
output logic [3:0] led          // R14, P14, N16, M14
);

logic [7:0] result;
logic      done_reg;
// FSM States
typedef enum logic [1:0] {IDLE=2'b00, CONVERT=2'b01, DONE=2'b10} state_t;
state_t state;
always_ff @(posedge clk) begin
    if (reset_btn) begin
        state    <= IDLE;
        result    <= 8'h00;
        done_reg <= 1'b0;
    end else begin
        case (state)
            IDLE: begin
                done_reg <= 1'b0;
                if (start_btn) state <= CONVERT;
            end
            CONVERT: begin
                // Load pre-computed SAR output based on SW[0]
                // SW[0]=0: Vin=11.2V -> 0xB3 = 8'b10110011
                // SW[0]=1: Vin=6.5V  -> 0x68 = 8'b01101000
                result <= sw[0] ? 8'h68 : 8'hB3;
                state  <= DONE;
            end
            DONE: begin
                done_reg <= 1'b1;
                if (!start_btn) state <= IDLE;
            end
            default: state <= IDLE;
        endcase
    end
end
// Output nibble selection based on SW[1]
// SW[1]=0: lower nibble; SW[1]=1: upper nibble
always_comb begin
    if (done_reg)
        led = sw[1] ? result[7:4] : result[3:0];
    else
        led = 4'b0000;
end
endmodule

```

Appendix 3: XDC Constraint File

A3.1 pynq_z2_sar_adc.xdc

Purpose: Pin constraint file mapping logical design ports to physical PYNQ-Z2 board pins.

```

## pynq_z2_sar_adc.xdc
## Xilinx Design Constraints for PYNQ-Z2 SAR ADC Demo

## Clock - 125 MHz system clock
set_property -dict {PACKAGE_PIN H16 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -period 8.000 -name sys_clk [get_ports clk]

```

```
## Buttons
set_property -dict {PACKAGE_PIN D19 IOSTANDARD LVCMOS33} [get_ports reset_btn]
set_property -dict {PACKAGE_PIN D20 IOSTANDARD LVCMOS33} [get_ports start_btn]

## Switches
set_property -dict {PACKAGE_PIN M20 IOSTANDARD LVCMOS33} [get_ports {sw[0]}]
set_property -dict {PACKAGE_PIN M19 IOSTANDARD LVCMOS33} [get_ports {sw[1]}]

## LEDs
set_property -dict {PACKAGE_PIN R14 IOSTANDARD LVCMOS33} [get_ports {led[0]}]
set_property -dict {PACKAGE_PIN P14 IOSTANDARD LVCMOS33} [get_ports {led[1]}]
set_property -dict {PACKAGE_PIN N16 IOSTANDARD LVCMOS33} [get_ports {led[2]}]
set_property -dict {PACKAGE_PIN M14 IOSTANDARD LVCMOS33} [get_ports {led[3]}]
```